ELSEVIER

Information and Software Technology 45 (2003) 539–546

INFORMATION AND SOFTWARE TECHNOLOGY

www.elsevier.com/locate/infsof

Code and data spatial complexity: two important software understandability measures

Jitender Kumar Chhabra[a,*], K.K. Aggarwal[b], Yogesh Singh[c]

[a]Department of Computer Engineering, National Institute of Technology (Deemed University), Kurukshetra 136119, India
[b]Vice-Chancellor, GGS Indraprastha University, Delhi 110006, India
[c]School of Information Technology, GGS Indraprastha University, Delhi 110006, India

Received 5 October 2002; revised 22 February 2003; accepted 3 March 2003

Abstract

In order to maintain the software, the programmers need to understand the source code. The understandability of the source code depends upon the psychological complexity of the software, and it requires cognitive abilities to understand the source code. The individual needs to correlate the orientation and location of various entities with their processing, which requires spatial abilities. This paper presents two measures of spatial complexity, which are based on two important aspects of the program—code as well as data. The measures have been applied to 15 different software projects and results have been used to draw many conclusions. The validation of the results has been done with help of perfective maintenance data. Lower values of code as well as data spatial complexity denote better understandability of the source code.
© 2003 Elsevier Science B.V. All rights reserved.

Keywords: Code spatial complexity; Data spatial complexity; Understandability; Software metrics; Psychological complexity

1. Introduction

In order to control, manage, and maintain software, the software complexity needs to be measured. If you cannot measure it, you cannot control it [7]. There are many aspects of the software. Some of them contribute towards the design and algorithmic complexity, some contribute towards readability and understandability of the software, and some other aspects have an influence on the debugging and testability of the software. No single metric of complexity is adequate to indicate all of these aspects of the software [9]. For example McCabe's cyclomatic complexity is a measure of control flow complexity [11], Halstead's science metrics concentrate on size of the software, average number of live variables per statement and program weakness indicate the design complexity [2, 13]. But these types of metrics do not indicate the complexity related to understandability and readability of the software. Understandability of the software is very important from maintenance point of view. The more understandable the source code is, the more quickly and accurately a programmer can obtain critical information about a program by reading the code. Increased understanding also leads to better management of software projects. Some measures of understandability of software-documents have been proposed [10,12], but these are not directly applicable to source code, as the understandability of the source code demands the knowledge of the corresponding programming language also. This type of complexity is related to psychological complexity. A program that is psychologically complex is difficult to understand. The theory of working memory is very useful to measure psychological complexity and directly affects the understandability of source code [3]. Spatial measures proposed below are based on this theory of working memory.

Spatial ability is a term that is used to refer to an individual's cognitive abilities relating to orientation, the location of objects in space, and the processing of location related visual information. Spatial ability has been correlated with the selection of problem solving strategy, and has played an important role in the formulation of an influential model of working memory [8]. In order to debug and

* Corresponding author. Tel.: +91-1744238678; fax: +91-1744238050.
E-mail addresses: aggarwal_krishan@hotmail.com (K.K. Aggarwal), ys66@rediffmail.com (Y. Singh), jitenderchhabra@rediffmail.com (J. Kumar Chhabra).

0950-5849/03/$ - see front matter © 2003 Elsevier Science B.V. All rights reserved.
doi:10.1016/S0950-5849(03)00033-8

maintain the software, programmer must understand the code, have an understanding of the application domain, and establish an appreciation of the relationships that can exist between the two [4]. Program comprehension and software maintenance are considered to substantially use programmers' spatial abilities. The software can be better understood, if one is able to correlate call to main components with their respective definitions.

Every software consists of two parts: code and data. To understand the behaviour of any software, one needs to comprehend both of these entities. The program's code helps in understanding the processing logic and the data variables and constants help in recognizing the input and output of the software. The spatial complexity based on the code is dependent on the definition and use of various components of the software. But there are many software, which handle lots of data and do comparatively lesser processing. In those types of software, the spatial complexity of the software is heavily dependent on data, rather than on code. So one needs to take into consideration both aspects (code as well as data) while computing the spatial complexity. The overall spatial complexity of software is integration of code-spatial and data-spatial complexity. The spatial complexity measure introduced in Ref. [8] is silent about the impact of data towards spatial abilities. They have not considered the data spatial complexity at all, and have defined only one function complexity metric. Even for that metric also, neither any empirical results have been produced nor any validation of the metric has been performed. In this paper we have proposed two spatial complexity measures based on code as well as data. The empirical results for both of these metrics are also produced below and validation of the metrics has been done with help of perfective maintenance data.

To compute the code-spatial complexity, the module is considered as the basic unit, as every module is defined at one place, but is called many times. The functionality of the module is visible in the definition part, while the use of that module is understood through various calls of that module. Thus the programmer needs to establish a relation between every module's definition and its uses in order to understand the processing logic of the software. This concept of module definition and uses is used below in part 2 to define and compute the code-spatial complexity. Similarly, every constant and variable is defined once, but is used at many places throughout the software. All inputs, intermediate results, and outputs are represented through variables or constants. Different variables and constants are processed at multiple places to generate the final output. The functionality of any software is dependent on the inputs, the use of various processing steps to generate the intermediate results, and then ultimately computing the final outputs. So these variables and constants are used to compute the data-spatial complexity, as explained in part 3 of the paper.

2. Code spatial complexity

Every software is written using various modules. A module basically means a function/procedure/sub-program in any language. The processing details of software can be understood by interrelating the definition of every module with its corresponding uses. The greater the distance in lines of code between the definition and use of the modules, more is the cognitive effort required to comprehend the connections of those modules in the software [8]. If a module is being called immediately after its definition, the understanding will be easier as no searching for that module is to be done and the details of that module are present in the working memory of the human being. On the other hand, if a module is called after 1000 lines of its definition, lot of searching/thinking has to be done, as many modules details appearing in those 1000 lines will get their place in the working memory of the human mind, and recalling the details of a module read 1000 lines earlier may not be easy. Thus the code-spatial complexity of a module (MCSC) is defined as average of distances (in terms of lines of code) between the use and definition of the module i.e.

$$\mathrm{MCSC} = \frac{\sum_{i=1}^{n} \mathrm{Distance}_i}{n}$$

where n represents count of calls/uses of that module and $\mathrm{Distance}_i$ is equal to the absolute difference in number of lines between the module definition and the corresponding call/use. The distance can be easily computed as long as the module call and definition belong to same file. Many a times source code of the software is written in multiple files. In such cases, a module may be defined in one file and used/called in some other file. The above definition of distance may be incomplete in those situations. When a module is called in a file, where it is not defined, the programmer first tries to find that module in the current file, and then looks for that module's definition in the other file. Thus understanding of such calls takes more cognitive efforts. The efforts are dependent on the file in which the module is being called/used and the file in which it is defined. In that case, we define the distance for that particular call of the module as summation of the distance of module call from bottom of the current file and the distance of module definition from top of that file (containing definition of the module).

Distance = (Distance of call from bottom of file containing call) + (Distance of definition of the module from top of the file containing definition)

Total code-spatial complexity of a software is defined as average of code-spatial complexity of all modules

$$\mathrm{CSC} = \frac{\sum_{i=1}^{m} \mathrm{MCSC}_i}{m}$$

where m is count of number of modules in the software.

3. Data spatial complexity

All input, intermediate, and final output values are an important part of the software, which cannot be ignored. The functionality of the software can be easily understood, if the programmer is able to comprehend the input and output. The processing is applied to input, and after many intermediate values, the final output gets generated. So in order to understand the full functionality of the software, all input, intermediate, and output values should be recognized. These values are represented through variables and constants in the software. So the cognitive efforts needed to understand the software also depend on the variables and constants, which is being considered in this paper for the first time in the literature. These efforts are measured in terms of data-spatial complexity. The data-spatial complexity gets affected by global as well as local variables and constants. The method to compute the data-spatial complexity differs from measurement technique of code-spatial complexity. The basic purpose of every data member is to store some value, which will be processed many times to generate some final results. To understand the purpose of any data member, various changes taking place on that member need to be understood. Every data member gets many changes in a particular sequence. That sequence also contributes towards the cognitive efforts needed to understand the purpose of that particular data member. Data member definition is not as important as was the module definition. The data definition does not tell anything else than its data type (and may be initial value). More details about the data member are understood through its uses in a particular sequence. When a data member is used for the first time, its initial value may be of use, but after that if the data member is used again, the previous value of the data member is more important than its initial value. If a data member is being used at some place, understanding the processing being done at that place is dependent on the previous use of that data member, instead of its original definition. So the data spatial complexity of any data member is measured using the distance between two successive uses, while code spatial complexity of a module is measured using the definition of that module always. The greater the distance in lines of code between the successive uses of the data members, more is the cognitive effort required to understand the purpose and data flow of that data member. If a data member is successively used at very small intervals, the details about that data member remain in the working memory of the programmer, and thus he will be able to comprehend that use of the data member easily. On the other hand, if a data member is used after say 1000 lines of its previous use, the programmer is very likely to have forgotten the details of that data member, and it will take more effort to establish a connection between the current use of the data member with its previous use. This concept of the data spatial complexity very closely resembles with average span of a variable, which has been already accepted as a good complexity measure [2,6,13]. Based on the above discussion, we define the data-spatial complexity of a data member (DMSC) as the average of distances of various uses of that data member from their previous use

$$\mathrm{DMSC} = \frac{\sum_{i=1}^{p} \mathrm{Distance}_i}{p}$$

where p represents count of uses of that data member and $\mathrm{Distance}_i$ is equal to the absolute difference in number of lines of the current use of the data-member from its just previous use. If a data member is being used for the first time, then the distance is defined as absolute difference (in lines of code) of the current use from the definition of the data member. If the data member is defined and used in the same source-code file, the distance can be calculated as above, but if the software is written using multiple source-code files, then a data member may be defined in one file and used in some other file. In that case, the above definition of distance will be incomplete. When a data member is used for the first time in a file, where it is not defined, the programmer first tries to find that data member in the starting of the current file, because the data members are usually defined at the start of any file/module. If the definition is not present in the current file, the programmer tries to find the details of that data member in the other file. Thus understanding of such uses takes more cognitive efforts. The efforts are dependent on the file in which the data member is being used and the file in which it is defined. In that case, we propose the distance for that particular use (first use only) of the data member as summation of the use of the data member from top of the current file and the distance of the data member definition from top of the new file (containing definition of the data member).

Distance = (Distance of first use of the data member from top of current file) + (Distance of definition of the data member from top of file containing defn.)

Usually, the local data members are used for storing some temporary values, and global data members store relatively permanent data. So it is expected that global variables and constants will contribute more towards data-spatial complexity that local variables and constants. The authors have collected results of data spatial complexity of all global as well as local data members of 15 different projects, which varied in size from 200 LOC to 3700 LOC

and it has been observed that effect of local data members on data spatial complexity is much lesser as compared to global data. One of the reasons for this is a very limited scope of local data. All local data members have their scope and use within a single module, whose length on average is 50–100 lines, which means that distance between their successive uses will be very few lines always and thus will be very easy for human memory to remember. Moreover if all local data members of all modules are also considered along with global data, then total count of data members goes in thousands, which overpowers the effect of global data while computing data spatial complexity. Based on these facts, it has been found by authors that only global data members need to be considered for computing data spatial complexity. Thus total data-spatial complexity of the software (DSC) is defined as average of data-spatial complexity of all global data members (variables as well as constants)

$$\mathrm{DSC} = \frac{\sum_{i=1}^{q} \mathrm{DMSC}_i}{q}$$

where q is count of global variables and constants in the software.

4. Applicability of spatial complexity to object-oriented software

The concept of the spatial complexity can be useful to measure the understandability of the object-oriented software as well. The main difference in procedure-oriented and object-oriented software comes from the usage of classes and objects. A class consists of code as well as data, and both have significant effect on working and understandability of the class. So the measurement of code spatial complexity and data spatial complexity should be quite useful to measure the cognitive efforts needed to understand the object-oriented software. But we feel that these definitions of code and data spatial complexity may not be directly applicable to the object-oriented software, as these two measures do not try to establish the linkage between the data members (attributes) and code-members (methods) of a class. The proposed measures consider the code and data separately in the software, but code and data are encapsulated in form of a class in object-oriented software. Hence these two measures need to be modified to take care of encapsulation and at the same time some new metric of spatial complexity may be needed to measure the interaction of the objects being used at various places of the object-oriented software.

5. Results and discussions

We applied these measures on 15 different projects of under-graduate and post-graduate level, which varied in size as well as functionalities and computed their code as well as data spatial complexities, which are summarized in Table 1. Out of these 15 projects, project number 10 (design of an editor) and project number 14 (design of a compiler) were developed by following the software engineering practices (meaning that software life cycle was strictly followed and software was designed keeping in mind low coupling and high cohesion, with an aim of producing a maintainable and quality software) and other projects were picked up randomly. The code-spatial complexity and data-spatial complexity are plotted against the lines of code and the corresponding plots are shown in Fig. 1. The concept of code-spatial complexity and data spatial complexity can be used in following ways.

1. The value of code-spatial complexity gives a hint about the effort needed to understand the processing logic of the source-code. Higher value of CSC means more cognitive efforts needed to understand the processing logic

Table 1
CSC and DSC values of different projects

S.N	Lines of code (LOC)	No. of modules	CSC	CSC/LOC	No of global variables and constants	DSC	DSC/LOC
1	237	9	76.59	0.32	6	82.67	0.35
2	500	7	152.5	0.31	1	7.6	0.02
3	532	7	164	0.31	1	7.36	0.01
4	576	13	79.15	0.14	11	44.56	0.08
5	636	13	277.7	0.44	12	52.5	0.08
6	641	10	326.8	0.51	6	135.7	0.21
7	646	16	160.62	0.25	8	97.03	0.15
8	1239	21	368.23	0.30	31	133	0.11
9	1269	32	250.9	0.20	70	265.86	0.18
10	1950	32	290.63	0.15	25	78.62	0.04
11	2247	35	506.81	0.23	45	187.43	0.08
12	2469	45	407.23	0.16	42	203.67	0.08
13	2783	42	549.1	0.20	78	368.17	0.13
14	3146	57	421.57	0.13	37	198.78	0.06
15	3682	46	609.23	0.17	83	392.86	0.11

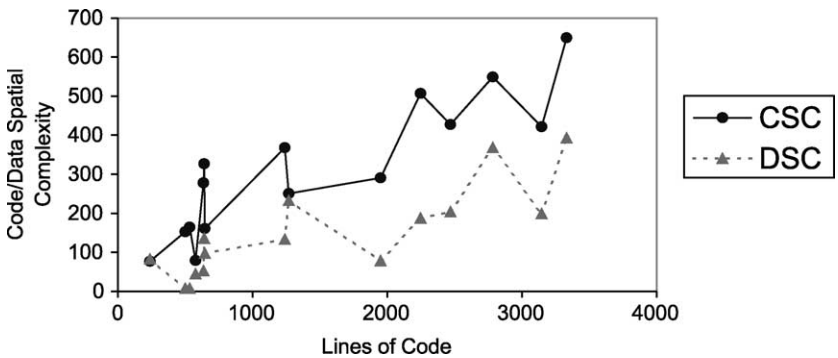

Fig. 1. Code and data spatial complexity versus lines of code.

of source code. In order to verify our intuition, we applied perfective maintenance on all of these 15 projects. In order to achieve uniformity of the operation, a target of improving the time efficiency of every project by about 10 percent was decided. Fifteen teams (comprising of 1–3 programmers) were assigned one project each and each of them was asked to identify those components of the software, where the improvement in the efficiency was possible and suggest the changes that could be applied in order to achieve the improvement in efficiency. This activity needed a thorough understanding of working of the source code. The results collected for each of these projects are shown in Table 2. If the maint-time and CSC values are plotted against LOC, as shown in Fig. 2, then it can be observed that behaviour of maint-time is similar to CSC with respect to LOC of the projects. For example, the project number 15 has got the CSC value as 609.23, which is the largest one, and the maint-time value of this project is also highest. As the aim of the maintenance needed a good understanding of the source code, this project number 15 is likely to be most difficult to understand.

2. Usually longer programs will be more difficult to understand, provided other factors (like meaningful naming etc.) remain same during development. It can be observed from Table 2, where maintenance time of last 4 projects (having 2500–3700 LOC) is much more than maintenance time of first 7 projects having loc up to 1000. Hence average code-spatial complexity is likely to increase with increase in lines of code, because more number of lines of code are likely to increase the distance between call and definition of modules. But the length of the software cannot be easily controlled by a great extent. The bigger projects will definitely have more lines of code, and thus will require more effort to understand. The understandability of the software needs to be measured within the constraints of the length. One smaller software may have lesser understandability than a bigger software. So the percentage of increase in code-spatial complexity with respect to lines of code can be another important factor to find out the level of difficulty of understanding. This ratio can be used to compare the understandability of two programs of different length, keeping aside the length factor and is plotted in Fig. 3. For example in the Table 1, if we compute the ratio of CSC to LOC, we find that project number 6 (having 641 LOC) has got the highest ratio of CSC and LOC as 0.51, which means that on an average the function call differs from its calls by almost 50% lines of total lines. This reflects that this project has got comparatively more code-spatial complexity, and will be more difficult to read and comprehend. Our empirical data collected also strengthens our belief, where the value of maint-time/KLOC ratio for project numbers 5 and 6 is around 80, which is much higher than other projects. On the other hand the value of CSC/LOC ratio comes out to be 0.13–0.15 for project number 4 (LOC as 576), project number 10 (LOC as 1950) and project number 14 (LOC as 3146), and the value of maint-time/KLOC ratio for these projects is around 50 (the lowest ones), which clearly reflects that understandability of these software is much more than project number 6, although project number 10 and 14 are much longer in length. These results strengthen the belief of advantages of using software engineering principles throughout the development of the software, as project number 10 and 14 were designed and developed by strictly following these principles in order to generate the maintainable software. In spite of having very high value of lines of code and having comparatively

Table 2
Maintenance time of different projects

S.N	Lines of code (LOC)	Maintenance time (person-hrs)	Maint-Time (person-hrs)/KLOC
1	237	15	63.29
2	500	35	70.00
3	532	35	65.79
4	576	29	50.35
5	636	52	81.76
6	641	52	81.12
7	646	38	58.82
8	1239	70	56.50
9	1269	69	54.37
10	1950	96	49.23
11	2247	135	60.08
12	2469	153	61.97
13	2783	156	56.05
14	3146	155	49.27
15	3682	197	59.12

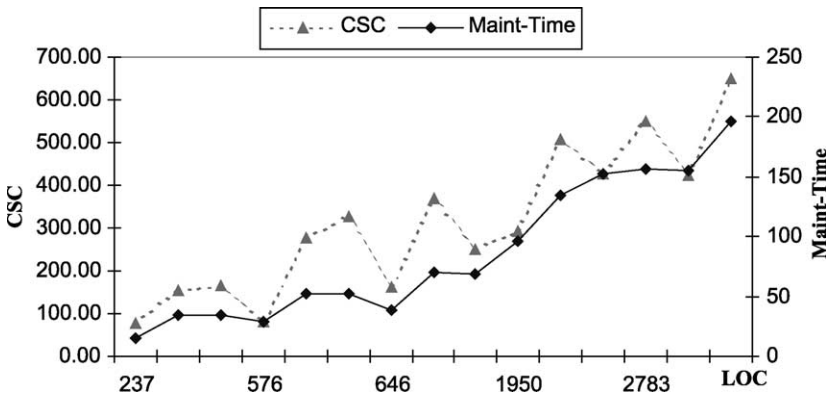

Fig. 2. CSC and Maint-Time versus LOC.

a difficult problem domain (design of compiler), project number 14 has got the one of the lowest values of the ratio of maint-time and KLOC. The plot of both ratios for all 15 projects is shown in Fig. 4. The similar behaviour of both ratios for these projects proves that CSC/LOC ratio can be a useful metric to compare the understandability of two programs.

3. Certain guidelines can be derived about the acceptable range of the code-spatial complexity and this range can be used by the software managers to judge the understandability of the software. But this will require a lot of analysis and statistics. If we look at the Fig. 3, then more than 50% of the software have got the ratio of CSC and LOC in the range of 0.2–0.3.

4. It may be noted that maint-time/ KLOC values have a strong correlation with CSC/LOC value. For almost all projects having CSC/LOC ratio value up to 0.30, the corresponding maint-time/KLOC value is in the range of 50–60. But if we look at the maint-time/KLOC value of project number 5 and 6, these values are unexpectedly high (81.76 and 81.12) in spite of comparatively small LOC. As explained above, the longer programs will have usually more maintenance time. This may be true when other factors (such as effective use of some techniques to produce quality software) remain similar. In that case, LOC may give some hint about the maintenance time. But when some special care is given in order to generate a maintainable and quality software (such as project number 10, and 14), then LOC may be a false indicator of the maintenance time. In those cases, LOC measure will not be able to take into the consideration the improvement in maintenance time because of improvement in understandability. LOC measure gives absolutely no clue about the readability of the source-code. Out of the 15 projects chosen, two projects (10 and 14) were developed with an aim of maintainable software, and LOC measure is not able to reflect this aspect at all. But CSC/LOC value is definitely able to do so. However, LOC value can be always used for normalization. Our results clearly indicate that the increase in maintenance time is not at all proportional to increase in LOC. However, it has a strong dependency on increase in CSC. These results get clearly reflected from Fig. 4, where correlation between the two lists is 0.92. In general, whenever we wish to estimate the understandability of the software, we have no idea whether some special techniques have been followed to produce the maintainable software or not. Thus it will be always better not to simply rely on LOC alone, but use the CSC/LOC ratio also. The lower value of CSC/LOC ratio is likely to help in reducing the maintenance time. For example, maint-time/KLOC values as well as CSC/LOC values of project number 4, 10 and 14 are very less as compared to other projects. So it is expected that if two projects differ a lot in their LOC size, then the LOC value may be sufficient to conclude that maintenance time of bigger software will be more. But how much more, may not be estimated without help of CSC/LOC measure. But every project manager would like to know the quantitative results of maintainability instead of just knowing that this project will require more (or less) time in maintenance as compared to some other project. Moreover, if the sizes of the two software are comparable, then CSC/LOC ratio may be very useful to compare their maintainability. This proposed metric is not simply an indicator of size, but it gives a clear indication of understandability of source-code, which has a direct impact on maintenance, and thus is useful in predicting the maintainability of the software.

5. The data spatial complexity value is very helpful in judging the appropriateness of global data. Higher value of data-complexity indicates that global variables have been used after a long gap of lines in the source code. It is always advisable not to use excessive global variables [5]. Improper choice of global variables will lead to less frequent usage of that global variable, and hence will add to data-spatial

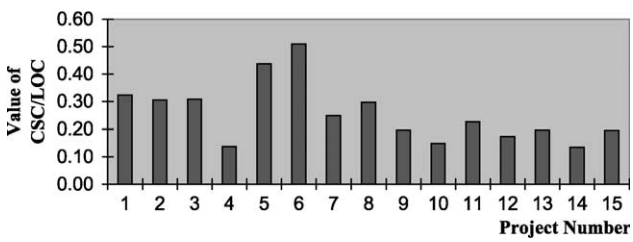

Fig. 3. Plot of CSC/LOC of all projects.

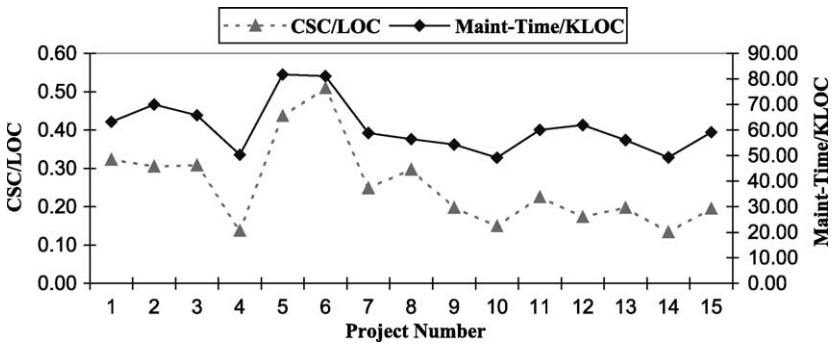

Fig. 4. Maint-Time/KLOC and CSC/LOC.

complexity. For example in the Table 1, project number 15 has highest data-spatial complexity as 392.86. This gives enough hints about improper choice of global data, which is clearly visible from the big count of 83 global data members. Very similar is the case of project number 13 (having 78 global data members) and project number 9 (having 70 global data members) with second and third highest data spatial complexity. These values of data-spatial complexity suggest that the global data members have not been properly chosen in this software.

6. As the length of the program increases, the total number of global variables and their use may increase (it can be observed from Table 1), and hence it may result in data-spatial complexity. But sometimes very irregular use of some global variables could exist because of wrong choice of global data. This effective utilization of global data can be found by calculating the ratio of DSC and LOC. These ratios are plotted in Fig. 5. For example in Table 1, this ratio value of project number 2 and 3 is 0.01–0.02 concluding the effective planning of global data. Both of these projects are comparatively smaller and contain only one global variable each. Project number 10 (of 1950 LOC) has got this ratio as 0.04, which means that all global variables have been very effectively used throughout the source code. Although the length of this project is 1950, which is approximately 9 times than LOC of project number 1, but the ratio of DOC/LOC of project number 10 is approx 1/9th of ratio value of project number 1. This result is on the expected lines, as this project was developed by following all software engineering principles, as already specified. Similarly, the ratio value of project number 14 is also very less (0.06 only) as compared to many other projects. This DSC/LOC metric is also not a simple measure of size, but it can be a very effective tool to find out the necessity of used global variables. If value of this metric comes out to be high, it gives enough hint of need of improving the software by reducing the global variables, which can be very helpful for adaptive maintenance activities.

7. It may be observed that the data spatial complexity increases as the number of global data increases. This will become evident by looking at the following graph in Fig. 6. It is clear from the graph that the data complexity increases as the number of global variables and constants increase. As minimum value of the data spatial complexity is always desirable, it further strengthens the guideline of using lesser global variables.

8. These spatial complexity values can help in judging the understandability of source code, which is an important factor in measuring the maintainability of the software [1]. Till now, the source code versus comment ratio has been used to compute the understandability of the source code [1, 14]. But now these values of code and data spatial complexity can be used to measure the understandability of the source code, which is used to measure the software maintainability.

6. Future works

In this paper, we have proposed two metrics of spatial complexity based on code as well as data. Our empirical results and data of perfective maintenance have given

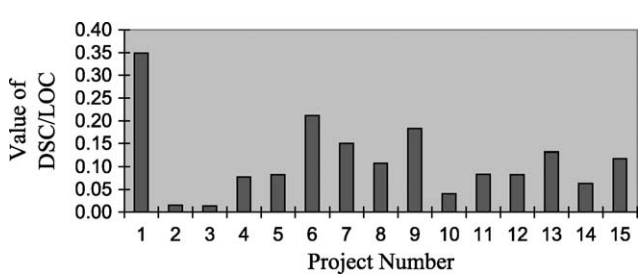

Fig. 5. Plot of DSC/LOC of all Projects.

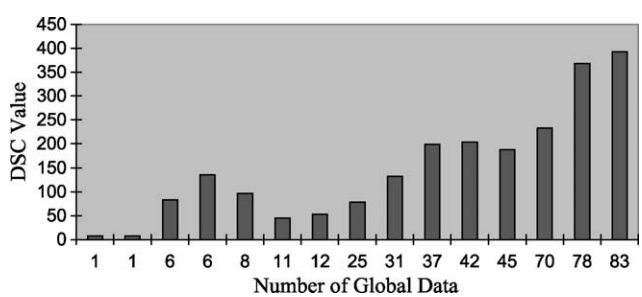

Fig. 6. Effect of global data on data spatial complexity.

enough indications about the possible utility of these two metrics. But our study has considered software of maximum 3700 lines of code. The effect of these metrics on understandability of the lengthy software (having size around 10000 or more lines of code) needs to be studied to strengthen the belief in these metrics. Moreover, relationship of these metrics with other types of maintenance such as corrective maintenance also needs to be considered. Secondly, we feel that code spatial complexity should be the domination factor in computation intensive software and data spatial complexity may be very dictating for data processing oriented application software, but this intuition needs to be validated empirically. Thirdly, the applicability of the spatial complexity metrics to the object-oriented software is a completely open area, where along with the modification of the proposed metrics, new metrics of spatial complexity may be proposed and their utilization in the understandability of object-oriented software can be analyzed.

7. Conclusion

The paper has proposed two measures of understandability of software: code spatial complexity and data spatial complexity. Code spatial complexity measures the efforts required to comprehend the definition of various modules with their uses. Data spatial complexity concentrates on measuring the efforts needed to grasp the purpose and uses of global variables and constants. The study of 15 different projects has been done and these spatial complexities have been measured for all of these projects. The results indicate that the adequate use of global data and software engineering practices helps in getting lesser value of these complexities, resulting into more understandable software. Lower value of code as well as data spatial complexity is always desirable for maintainable software.

References

[1] K.K. Aggarwal, Y. Singh, J.K. Chhabra, An Integrated Measure of Software Maintainability, Proceedings of Annual Reliability and Maintainability Symposium RAMS-2002—International Symposium on Product Quality and Integrity, Seatle Westin, USA, Jan 28–31, 2002, pp. 235–241

[2] K.K. Aggarwal, Y. Singh, J.K. Chhabra, Computing program weakness using module coupling, ACM SIGSOFT 27 (1) (2002) 63–66.

[3] A. Baddeley, Human Memory: Theory and Practice, Revised Edition, Hove Psychology Press, 1997.

[4] R. Brooks, Towards a theory of the comprehension of computer programs international, Journal of Man-Machine Studies 18 (1983) 543–554.

[5] J.K. Chhabra, Y. Singh, Software maintenance: write software that is easy to maintain, i.t, Information Technology Magazine 10 (7) (2001) 60–62.

[6] S.D. Conte, H.E. Dunsmore, V.Y. Shen, Software Engineering Metrics and Models, Cummings Pub Inc, USA, 1986.

[7] T. DeMarco, Controlling Software Projects, Yourdon Press, Englewood Cliffs, NJ, 1982.

[8] C.R. Douce, P.J. Layzell, J. Buckley, Spatial Measures of Software Complexity, Technical Report, Information Technology Research Institute, University of Brighton, UK, Jan 1999

[9] N.E. Fenton, S. Fleeger, Software Metrics—A Rigorous and Practical Approach, Thomson International Press, 2002.

[10] K. Laitnen, Estimating understandability of software documents, ACM SIGSOFT 21 (1996) 81–92.

[11] T.J. McCabe, A complexity measure, IEEE Transactions on Software Engineering SE-2 (4) (1976) 308–319.

[12] J.F. Peters, W. Pedrycz, Software Engineering: an Engineering Approach, Wiley, New York, 2000.

[13] Y. Singh, P. Bhatia, Module weakness: a new measure, ACM SIGSOFT 23 (5) (1998) 81–82.

[14] T. Tenny, Program readability: procedures versus comments, IEEE Transactions on Software Engineering 14 (9) (1988) 1271–1279.